

Multi-Corruption Image Restoration and Style-Conditioned Face-to-Sketch Generation: A Unified Framework with Autoencoder, Mixture-of-Experts, and Conditional GAN Architectures

Fahad Bin Faisal

Roll Number: i230614

Generative AI — Assignment #1

GitHub: <https://github.com/fhd-fsl/GenAI-A1>

Abstract—This paper presents the design, implementation, evaluation, and deployment of four generative AI systems for image restoration and synthesis. Tasks 1–3 address multi-corruption image restoration on the Oxford-IIIT Pet Dataset using progressively more sophisticated architectures: (1) a universal denoising autoencoder with a strict information bottleneck, (2) a corruption classifier with hard-routed specialist autoencoders, and (3) a jointly trained soft mixture-of-experts model with temperature-scaled gating. Task 4 addresses style-conditioned face-to-sketch generation on the FS2K dataset using a conditional generative adversarial network with a U-Net generator and PatchGAN discriminator. All models are trained in PyTorch with Optuna hyperparameter optimization (20 trials per task) and experiment tracking via Weights & Biases. The trained models are exported to ONNX format with verified numerical consistency ($< 10^{-5}$ maximum absolute difference), and served through a unified web application (a complete video demonstration is available at <https://youtu.be/CzVCmGIMKX8>) built with a React + Tailwind CSS frontend and a FastAPI backend, containerized and deployable via Docker Compose with a single command.

Index Terms—Image Restoration, Denoising Autoencoder, Mixture-of-Experts, Conditional GAN, Pix2Pix, ONNX, Optuna, Face-to-Sketch Synthesis

I. INTRODUCTION

Image restoration—the recovery of a clean image from its degraded observation—is a fundamental problem in computer vision with applications spanning medical imaging, autonomous driving, and digital photography [?]. The challenge is amplified when the nature of the degradation is unknown at inference time, requiring the model to implicitly identify and correct diverse corruption types within a single forward pass.

This work investigates three progressively more capable restoration paradigms. The *universal autoencoder* (Task 1) compresses the corrupted input through a strict information bottleneck, forcing the network to discard high-frequency noise and learn a shared latent representation across all corruption types. The *hard-routed specialist system* (Task 2) decomposes the problem by first classifying the corruption type and then routing the image to a dedicated restoration

expert, enabling per-corruption specialization at the cost of pipeline rigidity. The *soft mixture-of-experts* (Task 3) combines the benefits of both approaches by jointly training a differentiable gating network with the specialist experts, enabling continuous, input-dependent blending of expert outputs.

Additionally, Task 4 explores paired image-to-image translation for face-to-sketch generation using a conditional GAN based on the Pix2Pix framework [?]. The generator employs a U-Net architecture conditioned on categorical sketch-style embeddings, while a PatchGAN discriminator enforces local perceptual realism.

All four systems share a unified experimental methodology: PyTorch-based model development, Optuna-driven hyperparameter optimization with median pruning, experiment tracking via Weights & Biases (W&B), ONNX export with strict numerical verification, and deployment through a Dockerized React/FastAPI web application.

II. RELATED WORK

Denoising Autoencoders. Vincent et al. [?] introduced denoising autoencoders as a method for learning robust feature representations by training the network to reconstruct clean inputs from artificially corrupted versions. More recent convolutional approaches [?], [?] leverage residual learning and perceptual loss functions for high-fidelity image restoration. Our Task 1 architecture enforces a strict 1D bottleneck without skip connections, as mandated by the assignment, forcing the network to learn a genuinely compressed universal representation.

Mixture-of-Experts (MoE). The MoE paradigm, originally proposed by Jacobs et al. [?], partitions a complex problem across multiple specialist sub-networks. Shazeer et al. [?] demonstrated the effectiveness of sparse gating in large-scale language models. Our Task 3 implementation adapts this concept to image restoration, using temperature-scaled softmax gating with a balance regularizer to prevent routing collapse [?].

Conditional GANs for Image Translation. Isola et al. [?] proposed the Pix2Pix framework for paired image-to-image translation, combining a U-Net generator with a PatchGAN discriminator and an L1 reconstruction loss. Our Task 4 extends this framework with categorical style embeddings for multi-style face-to-sketch synthesis, following the conventions of the FS2K benchmark [?].

Structural Similarity (SSIM). Wang et al. [?] introduced SSIM as a perceptual quality metric that captures luminance, contrast, and structural information. We use SSIM as both a training loss component and an evaluation metric, weighted against L1 loss via an Optuna-tuned coefficient α .

III. DATASET PREPARATION

A. Oxford-IIIT Pet Dataset (Tasks 1–3)

The Oxford-IIIT Pet Dataset contains 7,349 images of cats and dogs across 37 breed categories. The category labels are not used for the restoration tasks; clean images serve as reconstruction targets. All images are converted to RGB and resized to 128×128 pixels.

The official training/validation collection is divided into 80% training (5,871 images) and 20% validation (1,468 images) using random seed 42. The official test set (3,669 images) remains untouched until final evaluation.

1) *Runtime Corruption Pipeline:* Corruptions are applied dynamically at runtime inside the data-loading pipeline. A custom `balanced_corruption_collate_fn` enforces strict per-batch class balance using modulo cycling ($\text{label} = i \bmod 4$) followed by random permutation, guaranteeing exactly 25% representation for each of the four conditions in every training batch:

- **Clean:** Original image without corruption.
- **Salt-and-pepper noise:** Pixel replacement with probability $p \sim \mathcal{U}(0.02, 0.15)$.
- **Gaussian blur:** Kernel size $k \in \{3, 5, 7\}$, standard deviation $\sigma \sim \mathcal{U}(0.5, 2.5)$.
- **Rectangular occlusion:** 1–3 black rectangles covering 10–35% of the image area.

2) *Deterministic Evaluation Manifests:* Validation corruptions are generated once and stored in a JSON manifest with fixed parameters. For the test set, each image receives 10 deterministic evaluations: 1 clean + 3 salt-and-pepper severities ($p \in \{0.05, 0.10, 0.15\}$) + 3 blur configurations ($(k, \sigma) \in \{(3, 1.0), (5, 1.5), (7, 2.5)\}$) + 3 occlusion levels ($\approx 10\%, 20\%, 35\%$ area), yielding 36,690 total test samples.

B. FS2K Dataset (Task 4)

The FS2K Facial Sketch Synthesis Dataset contains 2,104 paired photographs and sketches with three style categories. The official training set is subdivided into 85% training and 15% validation using stratified splitting by sketch style with seed 42. The official test set (1,046 images) remains isolated.

Paired data augmentation is applied during training: images are first resized to 144×144 , then identically random-cropped to 128×128 , horizontally flipped with 50% probability, and rotated by $\theta \sim \mathcal{U}(-10^\circ, 10^\circ)$. All spatial transforms are applied

identically to both the photograph and sketch to preserve pixel-level correspondence. Pixel values are normalized to $[-1, 1]$ for compatibility with the Tanh generator output.

IV. TASK 1: UNIVERSAL MULTI-CORRUPTION DENOISING AUTOENCODER

A. Architecture

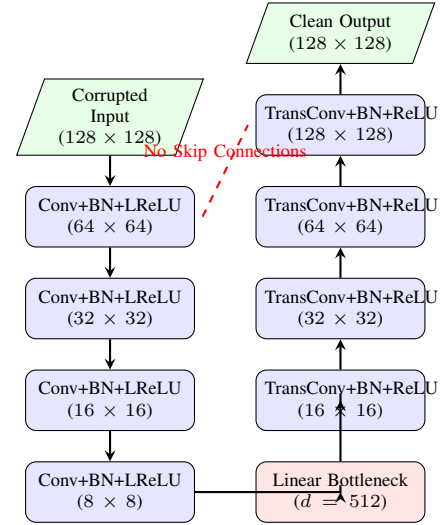


Fig. 1: Task 1: Universal Autoencoder Architecture.

The universal autoencoder consists of a convolutional encoder, a 1D latent bottleneck, and a convolutional decoder without skip connections.

Encoder. Four convolutional blocks progressively reduce spatial resolution: $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$, using 4×4 kernels with stride 2 and padding 1. Channel dimensions expand geometrically as $[C, 2C, 4C, 8C]$ where C is the base channel count. Each block applies Batch Normalization [?] followed by LeakyReLU(0.2). The $8 \times 8 \times 8C$ feature map is flattened and projected through a linear layer to a d -dimensional bottleneck vector. Dropout is applied before the linear projection.

Decoder. A linear layer maps the bottleneck vector back to the spatial dimensions $8C \times 8 \times 8$. Four transposed convolution blocks symmetrically expand the resolution: $8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$, with BatchNorm and ReLU activations. The final layer produces a 3-channel RGB output with a Sigmoid activation to constrain values to $[0, 1]$.

Design Justification. LeakyReLU is used in the encoder to prevent “dead neuron” problems during aggressive spatial compression, while standard ReLU suffices in the expanding decoder path. The strict 1D bottleneck—without U-Net-style skip connections—forces the network to discard high-frequency noise and learn a genuinely compressed representation. Transposed convolutions with kernel size 4 and stride 2 avoid checkerboard artifacts through exact divisibility [?].

B. Loss Function

The combined reconstruction loss is:

$$\mathcal{L}_{\text{task1}} = \alpha \cdot \mathcal{L}_{\text{L1}}(x, \hat{x}) + (1 - \alpha) \cdot (1 - \text{SSIM}(x, \hat{x})) \quad (1)$$

where α is a learnable trade-off parameter optimized by Optuna. L1 encourages pixel-accurate reconstruction, while the SSIM component preserves structural patterns, edges, and local contrast. The SSIM implementation uses `torchmetrics` with a data range of 1.0.

C. Hyperparameter Optimization

An Optuna study with 20 trials, using the Tree-structured Parzen Estimator (TPE) sampler and MedianPruner (5 startup trials, 5 warmup steps), searched the following space:

TABLE I: Task 1 Optuna Search Space and Best Configuration

Parameter	Search Range	Best Value
Learning rate	$[10^{-5}, 10^{-2}]$ (log)	1.13×10^{-4}
Batch size	{16, 32, 64}	32
Bottleneck dim	[64, 512] step 64	512
Base channels	{32, 48, 64}	48
Dropout	[0.0, 0.5]	0.026
Alpha (α)	[0.5, 1.0]	0.958

The best trial (#14) achieved a validation loss of 0.0900. The high $\alpha = 0.958$ indicates that the network heavily favored L1 reconstruction over SSIM, which is consistent with the observation that over-weighting SSIM can cause structural hallucinations in bottleneck autoencoders. Training used Adam optimization with early stopping (patience 5, $\delta = 0.001$) over a maximum of 30 epochs.

D. Experimental Results

The final model was evaluated on 36,690 deterministic test samples.

TABLE II: Task 1: Per-Corruption Results (Test Set)

Corruption Type	L1 ↓	SSIM ↑
Clean (baseline)	0.0701	0.5137
Gaussian Blur	0.0699	0.5126
Salt & Pepper	0.0712	0.5077
Rectangular Occlusion	0.0836	0.4715

TABLE III: Task 1: Per-Severity Results (Test Set)

Severity	L1 ↓	SSIM ↑
Low	0.0721	0.5061
Medium	0.0744	0.4984
High	0.0781	0.4872

Analysis. The model performs comparably on clean, blurred, and noisy inputs, demonstrating that the bottleneck architecture can effectively resolve local, high-frequency spatial degradations. Performance degrades most significantly for rectangular occlusion, which requires the model to semantically hallucinate entirely missing structural information—a

task beyond the capacity of a strict bottleneck autoencoder. As expected, L1 increases and SSIM decreases monotonically with corruption severity.

Failure Cases. The four worst-performing test instances (selected by the exact combined loss function) involve large rectangular occlusions covering critical facial features (e.g., eyes, snout), where the network produces visible blurring artifacts in the reconstructed region. Visual grids of 12 representative examples and 4 failure cases are included in the supplementary materials.

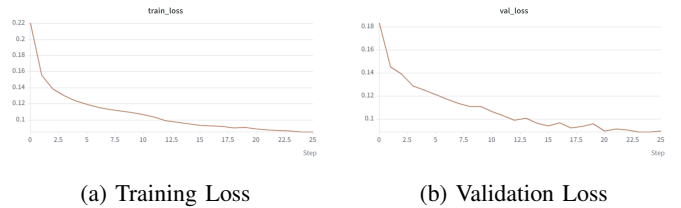


Fig. 2: Task 1: Combined (L1 + SSIM) Loss vs Epoch for the best Optuna trial (#14). Both curves demonstrate stable convergence without overfitting.

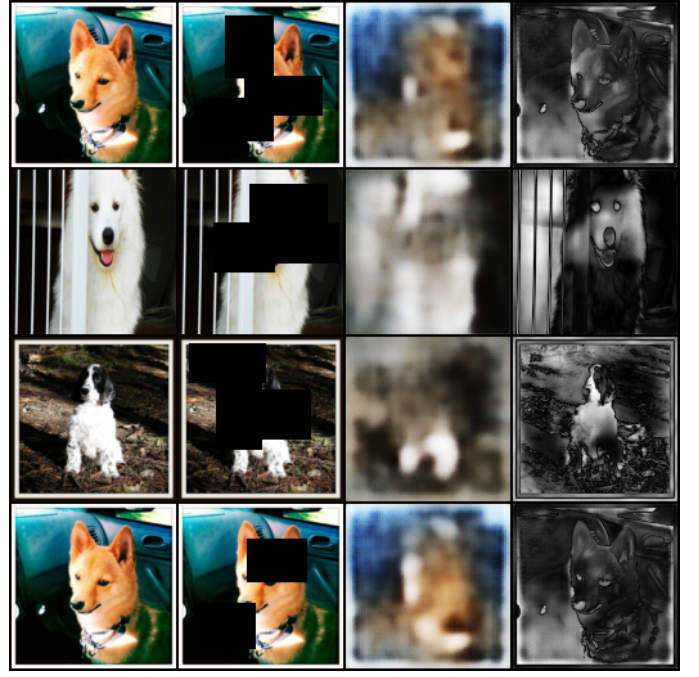
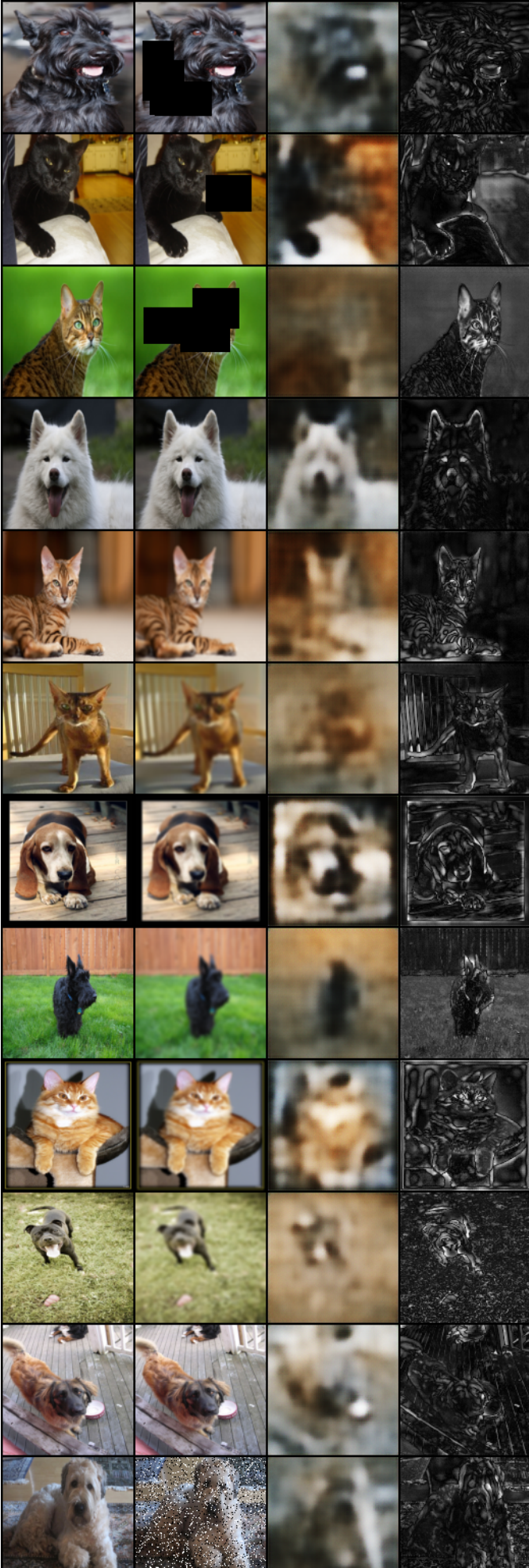


Fig. 4: Task 1: Failure cases with highest combined loss, predominantly involving large rectangular occlusions.

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALIST AUTOENCODERS

A. Corruption Classifier

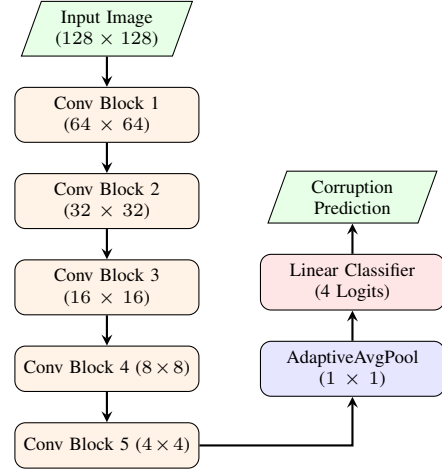


Fig. 5: Task 2: Corruption Classifier Architecture with Adaptive Pooling.

1) *Architecture*: The classifier uses 5 convolutional blocks that progressively halve spatial resolution ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$) using 3×3 kernels with stride 2 and padding 1, with channel expansion $[C, 2C, 4C, 8C, 8C]$. Each block applies BatchNorm and LeakyReLU(0.2). An Adaptive Average Pooling layer reduces the final feature map to 1×1 , making the classifier spatially invariant—critical for detecting rectangular occlusions regardless of mask position.

The classification head applies dropout followed by a linear projection to 4 logits (clean, salt-and-pepper, Gaussian blur, rectangular occlusion).

2) *Loss and Training*: The classifier is trained with cross-entropy loss using Adam with weight decay. The custom balanced collate function guarantees uniform class distribution in every batch.

TABLE IV: Task 2 Classifier: Optuna Search Space

Parameter	Search Range
Learning rate	$[10^{-5}, 10^{-2}]$ (log)
Batch size	{16, 32, 64}
Base channels	{16, 32, 64}
Dropout	[0.0, 0.5]
Weight decay	$[10^{-6}, 10^{-2}]$ (log)

3) *Hyperparameter Optimization*: Optuna was configured to minimize the validation cross-entropy loss, which provides a smoother optimization landscape than accuracy. 20 trials were executed with the best trial (#20) identified via early stopping with patience 5.

TABLE V: Task 2 Classifier: Test Set Metrics (N=36,690)

Metric	Value
Overall Accuracy	98.26%
Macro Precision	96.65%
Macro Recall	97.88%
Macro F1-Score	97.21%

TABLE VI: Task 2 Classifier: Per-Class F1-Scores

Corruption Class	F1-Score
Clean	91.89%
Salt & Pepper	99.98%
Gaussian Blur	99.41%
Rectangular Occlusion	97.57%

4) *Classification Results*: The classifier achieves near-perfect detection of artificial corruptions (>99% F1 for salt-and-pepper and blur). The slightly lower F1 for the “clean” class (91.89%) occurs because extremely subtle corruptions (e.g., blur with $\sigma = 0.5$) are occasionally misclassified as clean, which is an acceptable trade-off since it triggers the identity bypass rather than unnecessary restoration processing.

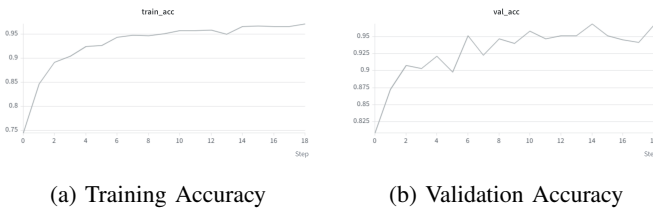


Fig. 6: Task 2: Classifier Accuracy vs Epoch. The model rapidly approaches near-perfect accuracy on both sets, indicating the spatial invariance (Adaptive Pooling) effectively captures corruption signatures.

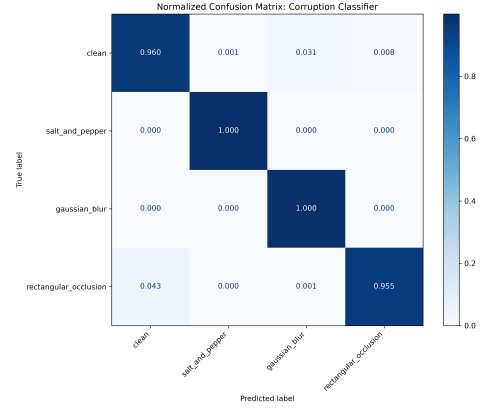


Fig. 7: Task 2: Normalized 4-class confusion matrix showing high diagonal accuracy and slight overlap between clean and low-severity corruptions.

B. Specialist Autoencoders

Three specialist autoencoders—one each for salt-and-pepper, Gaussian blur, and rectangular occlusion—reuse the UniversalAutoencoder architecture from Task 1 with independently trained parameters. A shared Optuna search (20 trials, 15 epochs each, 3 simultaneous micro-models per trial) identified the common architecture, after which each specialist was trained independently for up to 100 epochs to prevent VRAM exhaustion.

TABLE VII: Task 2 Specialists: Shared Optuna Search Space

Parameter	Search Range
Learning rate	$[10^{-5}, 10^{-2}]$ (log)
Bottleneck dim	[64, 512] step 64
Base channels	{32, 48, 64}
Batch size	{16, 32, 64}
Alpha (α)	[0.5, 1.0]

C. Hard-Routed Inference

During inference, the classifier first predicts the corruption type. If “clean” is predicted, the identity bypass returns the input unchanged. Otherwise, the corresponding specialist processes the image:

$$\hat{x} = \begin{cases} x_c, & r = \text{clean} \\ A_{\text{sp}}(x_c), & r = \text{salt} \\ A_{\text{blur}}(x_c), & r = \text{blur} \\ A_{\text{occ}}(x_c), & r = \text{occlusion} \end{cases} \quad (2)$$

TABLE VIII: Task 2: Hard-Routed Inference Results (N=36,690)

Routing Mode	L1 ↓	SSIM ↑
Oracle (ground truth)	0.06814	0.53327
Predicted (classifier)	0.06774	0.53699

Anomaly Analysis. Interestingly, predicted routing marginally outperforms oracle routing. This counterintuitive

result occurs because the classifier occasionally misroutes subtle corruptions (e.g., blur with $\sigma = 0.5$) to the identity bypass, which preserves the original sharp pixels better than forcing the image through a specialist that may over-smooth an already near-clean image.

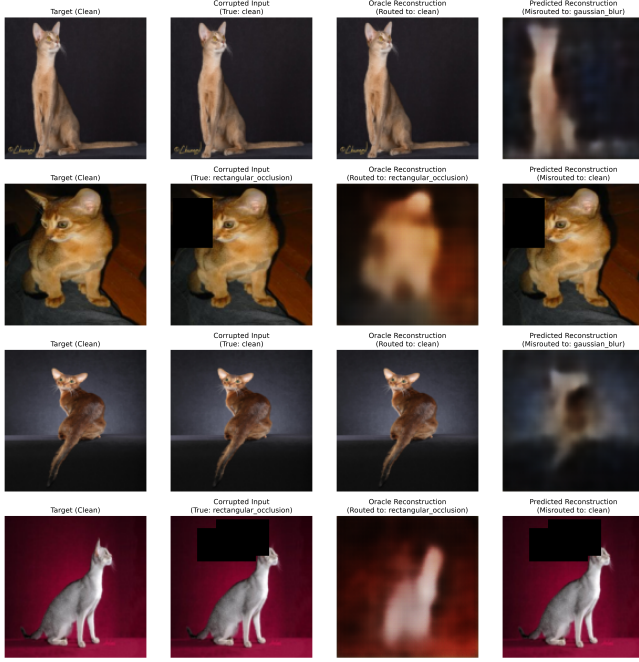


Fig. 8: Task 2: Cases where classifier misrouting produces visibly different outputs compared to oracle routing.

VI. TASK 3: JOINTLY TRAINED SOFT MIXTURE-OF-EXPERTS RESTORATION

A. Architecture

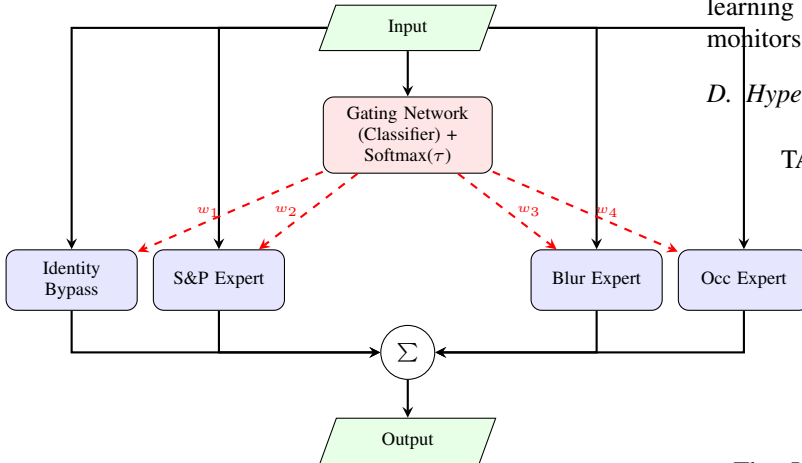


Fig. 9: Task 3: Soft Mixture-of-Experts Architecture.

The soft MoE model contains four branches: an identity pathway for clean images and three specialist autoencoders (initialized from Task 2). A gating network (initialized from

the Task 2 classifier) computes continuous routing weights via temperature-scaled softmax:

$$w = \text{softmax} \left(\frac{G(x_c)}{\tau} \right) \quad (3)$$

where τ is a non-learnable temperature parameter tuned by Optuna. The final reconstruction is:

$$\hat{x} = w_1 x_c + w_2 A_{\text{sp}}(x_c) + w_3 A_{\text{blur}}(x_c) + w_4 A_{\text{occ}}(x_c) \quad (4)$$

The temperature τ is registered as a buffer (not a learnable parameter) to prevent backpropagation from collapsing it to zero.

B. Joint Loss Function

$$\mathcal{L}_{\text{joint}} = \lambda_1 \mathcal{L}_{\text{L1}} + \lambda_2 (1 - \text{SSIM}) + \lambda_3 \mathcal{L}_{\text{CE}} + \lambda_4 \mathcal{L}_{\text{balance}} \quad (5)$$

The balance regularizer prevents routing collapse by penalizing deviation from uniform distribution:

$$\mathcal{L}_{\text{balance}} = \sum_{k=1}^4 \left(\bar{w}_k - \frac{1}{4} \right)^2 \quad (6)$$

where $\bar{w}_k$ is the mean routing weight for branch k within a batch. The cross-entropy component keeps the gating weights semantically aligned with the known corruption labels.

C. Two-Stage Training

Stage 1 — Warm-up (10 epochs): Expert parameters are frozen; only the gating network is trained using a dedicated warm-up learning rate. This prevents chaotic gradients from the untrained gate from destroying the pre-trained specialist weights.

Stage 2 — Joint Fine-tuning (≤ 100 epochs): All parameters are unfrozen and trained jointly with a smaller fine-tuning learning rate. Early stopping with patience 5 and $\delta = 0.0005$ monitors the validation reconstruction loss.

D. Hyperparameter Optimization

TABLE IX: Task 3 MoE: Optuna Search Space

Parameter	Search Range
Warm-up LR	$[10^{-5}, 10^{-3}]$ (log)
Fine-tune LR	$[10^{-6}, 10^{-4}]$ (log)
Temperature (τ)	$[0.1, 5.0]$
λ_{CE}	$[0.01, 0.5]$
λ_{balance}	$[10^{-3}, 0.1]$ (log)
λ_{recon}	$[0.1, 1.0]$
λ_{SSIM}	$[0.1, 1.0]$

The L1 and SSIM weights are treated as independent variables rather than a convex combination, providing the necessary mathematical freedom to balance pixel accuracy against structural quality on their different numerical scales. Custom trial pruning immediately terminates trials exhibiting routing collapse ($\max(\bar{w}) > 0.90$ or $\min(\bar{w}) < 0.05$).

E. Experimental Results

TABLE X: Task 3: Global Test Set Metrics (N=36,690)

Metric	Value
L1 Loss	0.05281
SSIM	0.70292
Avg Inference Time	0.14 ms/image

TABLE XI: Task 3: Per-Corruption Metrics

Corruption	Count	L1 ↓	SSIM ↑
Clean	3,669	0.00873	0.99043
Salt & Pepper	11,007	0.06947	0.52070
Gaussian Blur	11,007	0.04000	0.74703
Rect. Occlusion	11,007	0.06364	0.74518

The soft MoE achieves significantly better performance than both the universal autoencoder (Task 1: L1 = 0.074 avg) and the hard-routed system (Task 2: L1 = 0.068), reducing the overall L1 to 0.053 and improving SSIM from ~ 0.51 to 0.70. The clean image pathway achieves near-perfect reconstruction (SSIM = 0.990).

F. Routing Weight Analysis

TABLE XII: Task 3: Average Routing Weights by Corruption Type and Severity

Condition	Identity	S&P	Blur	Occ
Clean	0.872	0.007	0.038	0.083
S&P (low)	0.093	0.753	0.099	0.055
S&P (medium)	0.002	0.968	0.006	0.025
S&P (high)	0.000	0.980	0.000	0.020
Blur (low)	0.658	0.004	0.318	0.020
Blur (medium)	0.567	0.005	0.412	0.016
Blur (high)	0.560	0.006	0.415	0.020
Occ (low)	0.760	0.010	0.017	0.213
Occ (medium)	0.607	0.011	0.010	0.372
Occ (high)	0.427	0.010	0.007	0.556

Key Observations:

- 1) **Clean images** are routed predominantly to the identity branch ($w = 0.872$), confirming that the gate learns to bypass unnecessary processing.
- 2) **Salt-and-pepper** routing sharpens dramatically with severity: the S&P expert weight increases from 0.753 (low) to 0.980 (high).
- 3) **Gaussian blur** exhibits identity dominance even at high severity ($w_{\text{identity}} = 0.560$), suggesting the gate learned that the identity branch contributes useful high-frequency content that the blur expert alone cannot reconstruct.
- 4) **Occlusion** shows progressive transfer from identity (0.760 at low severity) to the occlusion expert (0.556 at high severity), demonstrating severity-adaptive routing.

- 5) All four experts remain active throughout evaluation, passing the inactivity check ($> 5\%$ total routing volume).

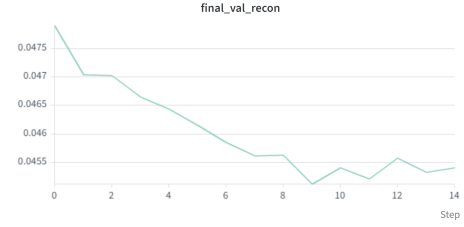


Fig. 10: Task 3: Validation Reconstruction Loss vs Epoch during the joint fine-tuning stage. The loss descends smoothly and plateaus, confirming stable joint expert training.

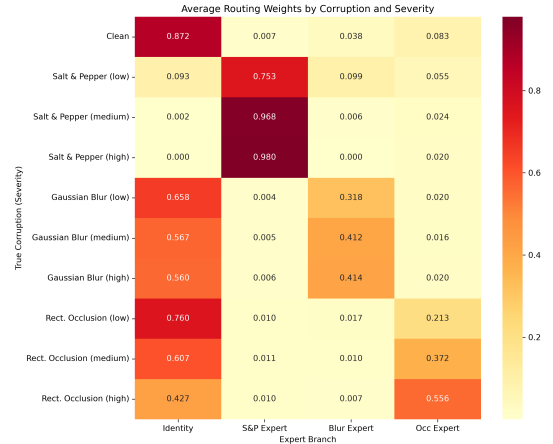


Fig. 11: Task 3: Routing weight heatmap across corruption types and severity levels.

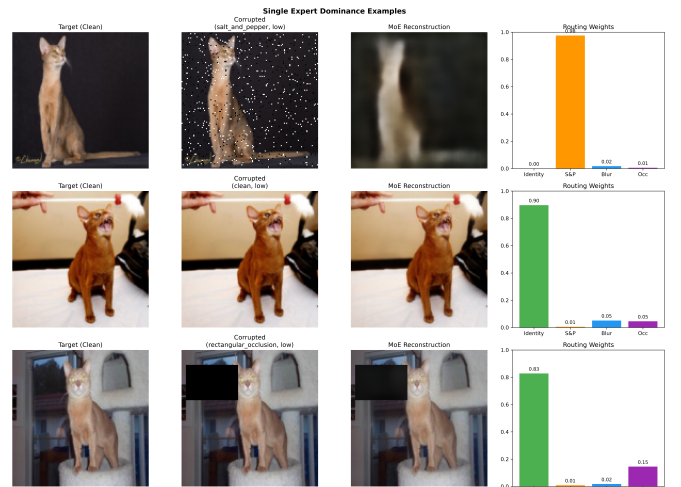


Fig. 12: Task 3: Examples where a single expert dominates the routing weights.

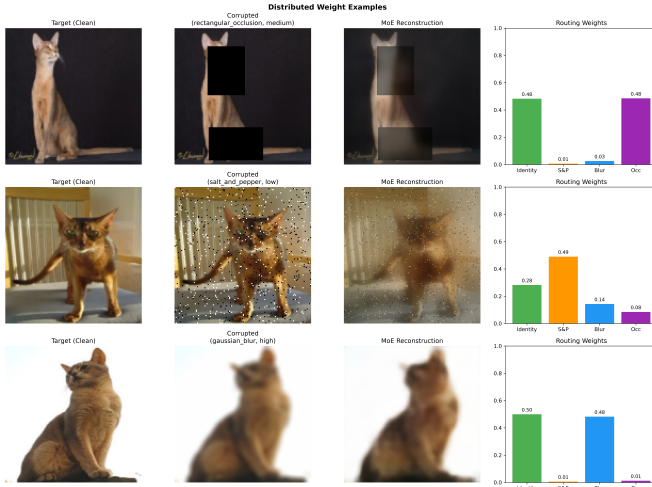


Fig. 13: Task 3: Examples where routing weights are distributed across multiple experts.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Architecture

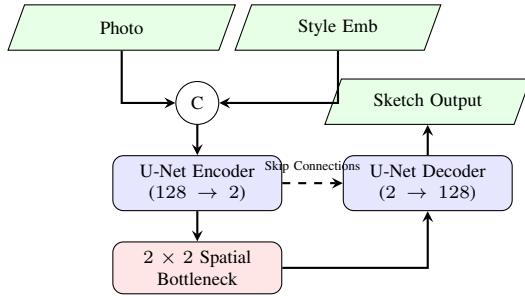


Fig. 14: Task 4: Conditional U-Net Generator Architecture.

1) *U-Net Generator*: The generator follows a 6-layer U-Net encoder-decoder architecture with skip connections. The style condition is incorporated via a learned categorical embedding ($\text{nn.Embedding}(3, d_e)$) that is spatially expanded to $d_e \times 128 \times 128$ and concatenated to the input photograph along the channel dimension, forming a $(3 + d_e)$ -channel input.

The encoder uses 4×4 strided convolutions ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$) with BatchNorm and LeakyReLU(0.2). The bottleneck retains a 2×2 spatial resolution rather than collapsing to 1×1 , preserving geometrically meaningful spatial layout information.

The decoder uses 4×4 transposed convolutions with BatchNorm, dropout (in the first two layers for regularization), and ReLU activations. Skip connections from the corresponding encoder layers are concatenated along the channel dimension before each decoder block. The output layer uses Tanh to produce values in $[-1, 1]$.

Design Justification. Using a learned embedding rather than a raw integer avoids imposing an artificial ordinal relationship between styles. The Tanh activation provides zero-

centered, symmetric outputs with stronger gradients than Sigmoid, mitigating vanishing gradient problems in deep adversarial networks [?].

2) *PatchGAN Discriminator*: The discriminator receives the concatenation of the input photograph, the sketch (real or generated), and the spatially expanded style embedding, forming a $(3 + 3 + d_e)$ -channel input. It applies 5 convolutional layers: three with stride 2 ($128 \rightarrow 64 \rightarrow 32 \rightarrow 16$) and two with stride 1 ($16 \rightarrow 15 \rightarrow 14$), producing a 14×14 grid of patch-level real/fake logits. This PatchGAN design evaluates local regions for realism rather than making a single global judgment.

B. Training Procedure

The generator objective combines adversarial and reconstruction losses:

$$\mathcal{L}_G = \mathcal{L}_{\text{BCE}} + \lambda_{\text{LI}} \cdot \mathcal{L}_{\text{LI}}(y, G(x, s)) \quad (7)$$

The discriminator is trained with standard binary cross-entropy:

$$\mathcal{L}_D = \frac{1}{2} [\mathcal{L}_{\text{BCE}}(D(x, y, s), 1) + \mathcal{L}_{\text{BCE}}(D(x, G(x, s), s), 0)] \quad (8)$$

Both networks use Adam with $\beta = (0.5, 0.999)$. A linear learning rate decay schedule keeps the rate constant for the first 50 epochs and linearly decays to zero over the remaining 50 epochs, stabilizing the adversarial equilibrium. Weight initialization follows $\mathcal{N}(0, 0.02)$ as established by the original Pix2Pix implementation.

Training logs record discriminator real/fake losses, generator adversarial/reconstruction losses, and validation metrics separately. Visual samples are logged to W&B every 5 epochs using a fixed validation batch.

C. Hyperparameter Optimization

TABLE XIII: Task 4 GAN: Optuna Search Space

Parameter	Search Range
Generator LR	$[10^{-5}, 10^{-3}]$ (log)
Discriminator LR	$[10^{-5}, 10^{-3}]$ (log)
Batch size	{8, 16, 32}
Base channels	{32, 64, 128}
Dropout	[0.0, 0.5]
Embedding dim	{8, 16, 32, 64}
λ_{LI}	[10.0, 200.0]

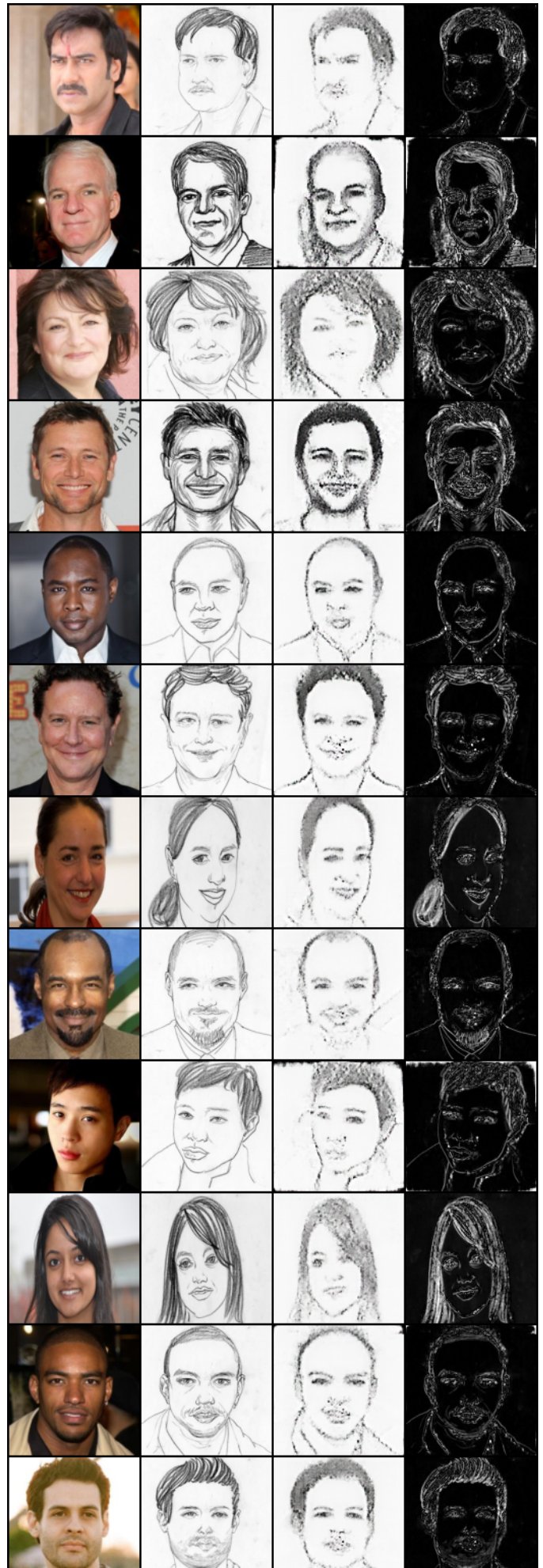
HPO trials used 15 epochs each due to GAN training expense. The best configuration was then retrained for the full 100-epoch schedule.

D. Experimental Results

TABLE XIV: Task 4: Per-Style Test Set Metrics

Style	Count	L1 ↓	SSIM ↑	PSNR ↑
Style 0	619	0.0806	0.5358	17.10
Style 1	381	0.1525	0.3926	12.42
Style 2	46	0.0642	0.6371	18.61

Style 2 achieves the best reconstruction metrics despite having the fewest training examples, likely because its visual characteristics (line-drawing style) are most amenable to pixel-wise comparison. Style 1, which involves more complex artistic transformations, presents the greatest challenge. The final checkpoint (epoch 100) was exported rather than the best-validation checkpoint because GAN validation loss, heavily influenced by the adversarial BCE component, is an unreliable proxy for perceptual quality.



(a) Validation Adversarial Loss (b) Validation L1 Recon Loss

Fig. 15: Task 4: Generator Validation Losses. The adversarial loss fluctuates as the discriminator and generator reach equilibrium, while L1 loss gradually decreases, ensuring structural fidelity.

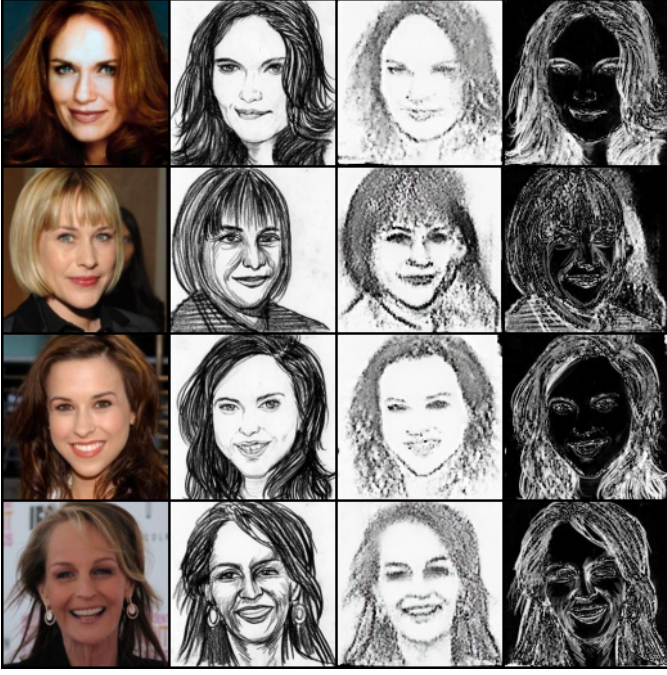


Fig. 17: Task 4: Failure cases with highest L1 error, typically involving complex backgrounds or unusual lighting.

VIII. COMPARATIVE ANALYSIS

TABLE XV: Cross-Task Restoration Performance Comparison (N=36,690)

System	L1 ↓	SSIM ↑
Task 1: Universal AE	0.0737	0.4984
Task 2: Hard-Routed (predicted)	0.0677	0.5370
Task 3: Soft MoE	0.0528	0.7029

The soft MoE demonstrates the clear advantage of differentiable, input-dependent routing over both the universal approach (28.4% L1 improvement) and hard routing (22.0% L1 improvement). The SSIM improvement is even more dramatic: from 0.498 (universal) to 0.703 (soft MoE), representing a 41% increase in structural fidelity. This is attributable to the soft system’s ability to blend expert outputs proportionally to corruption severity, maintaining high-frequency detail from the identity branch while leveraging specialist restoration when needed.

IX. ONNX EXPORT AND VERIFICATION

All models were exported to ONNX format using `torch.onnx.export()` with dynamic batch axes:

TABLE XVI: ONNX Model Inventory

Model	File Size	Opset
Universal AE	113 MB	17
Classifier	1.0 MB	17
Specialist (S&P)	72.5 MB	17
Specialist (Blur)	72.5 MB	17
Specialist (Occ)	72.5 MB	17
Soft MoE	218.6 MB	14
Generator	117.3 MB	17

Each export follows a strict verification pipeline:

- 1) Structural validity check via `onnx.checker.check_model()`.
- 2) Numerical consistency verification by comparing PyTorch and ONNX Runtime outputs on random input tensors.
- 3) All models pass with maximum absolute difference $< 10^{-5}$.

Inference in the application uses `onnxruntime.InferenceSession` with the `CPUExecutionProvider` for cross-platform compatibility.

X. APPLICATION ARCHITECTURE

A. Design

The application’s visual design was prototyped in Google Stitch to establish the layout, color palette, component hierarchy, and responsive behavior before implementation (see Fig. ??).

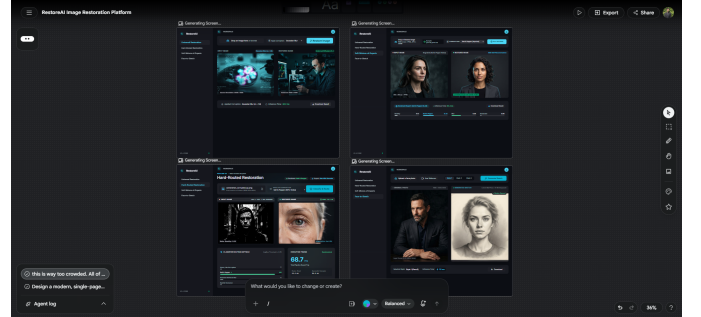


Fig. 18: Google Stitch design prototype for the application interface.

B. Frontend

The frontend is built with React and Tailwind CSS using Vite as the build tool. It provides four workspaces accessible via a sidebar navigation. A complete 3-minute video demonstration covering all application spaces and functionalities can be viewed here: <https://youtu.be/CzVCmGIMKX8>.

- **Universal Restoration:** Upload or select a clean image, apply corruptions via the interface, view the restored output with corruption settings and inference time.
- **Hard-Routed Restoration:** Displays classifier probabilities, predicted corruption, selected expert, reconstructed image, and timing breakdown.

- **Soft Mixture-of-Experts Restoration:** Shows all four routing weights, reconstructed result, inference time, and visual indication of dominant experts.
- **Face-to-Sketch Generator:** Upload a photo or capture via webcam, select Style 1/2/3, generate and download sketches.

C. Backend

The FastAPI backend provides five endpoint groups: health check, universal restoration, hard routing, soft mixture, and face-to-sketch generation. On startup, all seven ONNX models are loaded into memory via a singleton `ModelManager`. Each endpoint validates uploaded files, preprocesses images to the correct format (128×128 , NCHW, appropriate normalization), runs ONNX inference, and returns base64-encoded result images with timing information.

D. Deployment

The application is containerized with Docker Compose:

- **Backend container:** Python 3.12 slim, Uvicorn server on port 8000. ONNX models are mounted as read-only volumes.
- **Frontend container:** Multi-stage build (Node 20 Alpine → Nginx Alpine) serving the production bundle on port 80/3000.

A single `docker-compose up --build` command starts the complete application.

XI. LIMITATIONS AND FUTURE WORK

- 1) **Resolution constraint:** All models operate at 128×128 . Higher-resolution restoration would require progressive architectures or patch-based processing.
- 2) **Occlusion inpainting:** The strict bottleneck autoencoder fundamentally cannot hallucinate semantically plausible content for large missing regions. Diffusion-based or transformer-based inpainting would be more appropriate.
- 3) **Identity dominance in Soft MoE:** The gating network over-relies on the identity branch for Gaussian blur inputs, suggesting that the blur expert’s reconstruction does not sufficiently exceed the quality of the raw input. Curriculum-based training or per-expert auxiliary losses could address this.
- 4) **Style imbalance in FS2K:** Style 2 has only 46 test images, making its metrics statistically less reliable. Few-shot learning or style augmentation could improve robustness.
- 5) **GAN evaluation:** L1 and SSIM are imperfect metrics for generative quality. Fréchet Inception Distance (FID) would provide a more perceptually meaningful evaluation.

XII. CONCLUSION

This work demonstrates a systematic progression from a monolithic universal autoencoder to a differentiable soft mixture-of-experts system, achieving a 28.4% reduction in L1 loss and a 41% improvement in SSIM on the Oxford-IIIT

Pet corruption restoration benchmark. The corruption classifier achieves 98.26% accuracy with per-class F1 scores exceeding 97%, and the MoE routing analysis reveals interpretable, severity-adaptive expert allocation. The conditional GAN successfully generates style-conditioned facial sketches, with the full pipeline deployed as a single-command Dockerized web application. All models maintain ONNX numerical consistency below 10^{-5} tolerance, and every critical hyperparameter was systematically investigated through Optuna studies with 20 trials per task, tracked via Weights & Biases.

APPENDIX A AI-USE DISCLOSURE

The following AI tools were used during the development of this project:

- **Google Antigravity (Gemini-based coding assistant):** Used for code scaffolding, debugging, architecture prototyping, and report drafting. All generated code was manually verified, tested with custom unit tests, and modified to meet specific assignment requirements. Training scripts, evaluation pipelines, and ONNX exports were validated end-to-end before inclusion.
- **Google Stitch:** Used for UI/UX design prototyping of the application interface.
- **Weights & Biases:** Used for automated experiment tracking, loss curve visualization, and hyperparameter comparison across Optuna trials.

All AI-generated outputs were critically evaluated, tested against expected behavior, and corrected where necessary. The student maintains full understanding of every component in the submitted system.

REFERENCES

- [1] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising,” *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [2] P. Vincent, H. Larochelle, Y. Bengio, and P. A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proc. ICML*, 2008, pp. 1096–1103.
- [3] X. Mao, C. Shen, and Y. Yang, “Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections,” in *Proc. NIPS*, 2016, pp. 2802–2810.
- [4] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
- [5] N. Shazeer, A. Mirzadeh, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [6] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. CVPR*, 2017, pp. 1125–1134.
- [8] Y. Fan, L. Liu, F. Chen, Y. Luo, and K. Zhang, “FS2K: A facial sketch synthesis dataset,” in *Proc. AAAI*, 2022.
- [9] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [10] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015, pp. 448–456.
- [11] A. Odena, V. Dumoulin, and C. Olson, “Deconvolution and checkerboard artifacts,” *Distill*, 2016.